Name: Yarra Pawan
Reg No: 192424294
Course Code: DSA0503
Course Name: Query Processing for Data Science
Assessment: 2

1. **Hospital Doctor Scheduling (10 Marks)**

A hospital stores doctor details in XML format and appointment schedules in CSV format. Write a Python program to identify scheduling conflicts and generate an optimized appointment schedule
.

Aim

To read doctor details from an XML file and appointment schedules from a CSV file, identify scheduling conflicts, and generate an optimized appointment schedule using Python.

Algorithm

1. Start the program.
2. Import the Pandas library.
3. Read the doctor details from the XML file.
4. Read the appointment details from the CSV file.
5. Display the doctor and appointment details.
6. Check for duplicate appointments based on Doctor ID, Date, and Time.
7. Display the scheduling conflicts.
8. Remove duplicate appointments to create an optimized schedule.
9. Display the optimized appointment schedule.
10. Stop the program.

Code:

```
import pandas as pd
doctors = pd.read_xml("doctors.xml")
appointments = pd.read_csv("appointments.csv")
print("Doctor Details")
print(doctors)
print("\nAppointments")
print(appointments)
conflicts = appointments.duplicated(subset=["DoctorID", "Date", "Time"], keep=False)
print("\nScheduling Conflicts")
print(appointments[conflicts])
```

```
optimized_schedule = appointments.drop_duplicates(subset=["DoctorID", "Date",
"Time"])
print("\nOptimized Appointment Schedule")
print(optimized_schedule)
```

Sample Input:

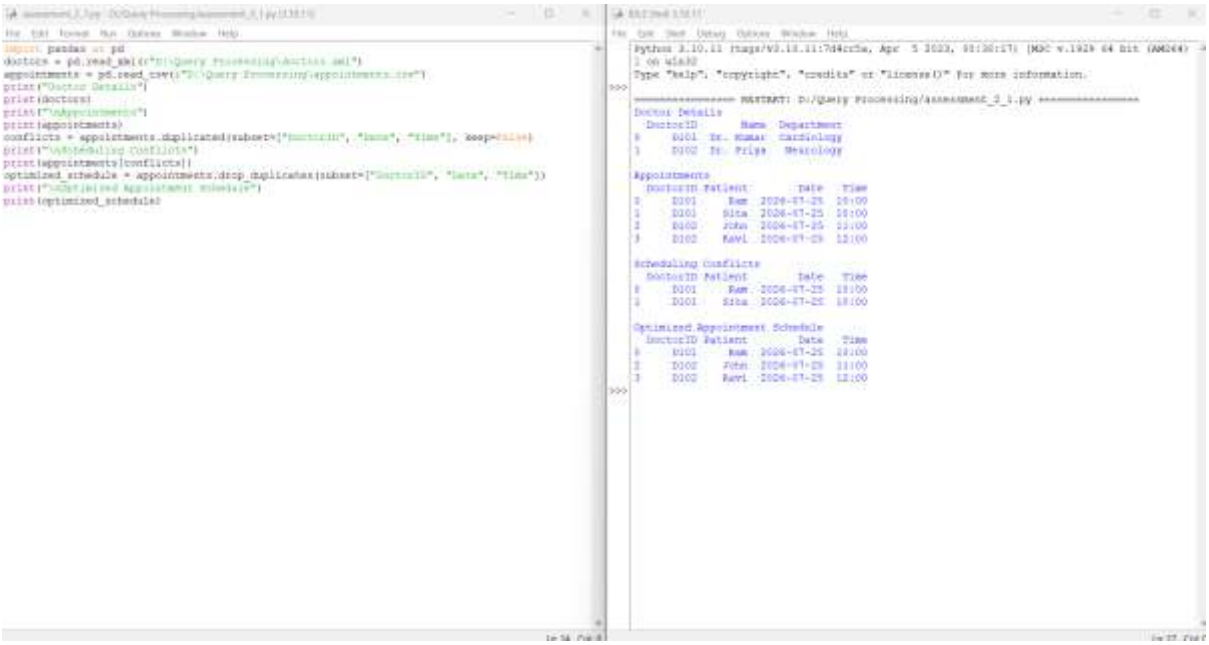
doctors.xml

```
<Doctors>
  <Doctor>
    <DoctorID>D101</DoctorID>
    <Name>Dr. Kumar</Name>
    <Department>Cardiology</Department>
  </Doctor>
  <Doctor>
    <DoctorID>D102</DoctorID>
    <Name>Dr. Priya</Name>
    <Department>Neurology</Department>
  </Doctor>
</Doctors>
```

appointments.csv

```
DoctorID,Patient,Date,Time
D101,Ram,2026-07-25,10:00
D101,Sita,2026-07-25,10:00
D102,John,2026-07-25,11:00
D102,Ravi,2026-07-25,12:00
```

Output:



Result: The program successfully reads doctor and appointment data, identifies scheduling conflicts, and generates an optimized appointment schedule.

2. **Airport Passenger Check-in System (10 Marks)**

An airport stores passenger details in CSV format and flight information in JSON format. Write a Python program to combine the datasets, verify passenger bookings, and generate boarding lists for each flight.

Aim

To read passenger details from a CSV file and flight information from a JSON file, combine the datasets, verify passenger bookings, and generate boarding lists using Python.

Algorithm

1. Start the program.
2. Import the Pandas library.
3. Read passenger details from the CSV file.
4. Read flight information from the JSON file.
5. Merge the passenger and flight datasets using Flight ID.
6. Verify passenger bookings by checking valid flight information.
7. Display the verified passenger details.
8. Generate and display the boarding list for each flight.
9. Stop the program.

Code:

```
import pandas as pd
passengers = pd.read_csv("passengers.csv")
flights = pd.read_json("flights.json")
print("Passenger Details")
print(passengers)

print("\nFlight Details")
print(flights)
merged = pd.merge(passengers, flights, on="FlightID", how="left")
print("\nCombined Data")
print(merged)
verified = merged[merged["Destination"].notna()]
print("\nVerified Passengers")
print(verified)
print("\nBoarding Lists")
for flight in verified["FlightID"].unique():
  print("\nFlight:", flight)
```

```
    print(verified[verified["FlightID"] == flight][["PassengerName"]])
```

PassengerID,PassengerName,FlightID

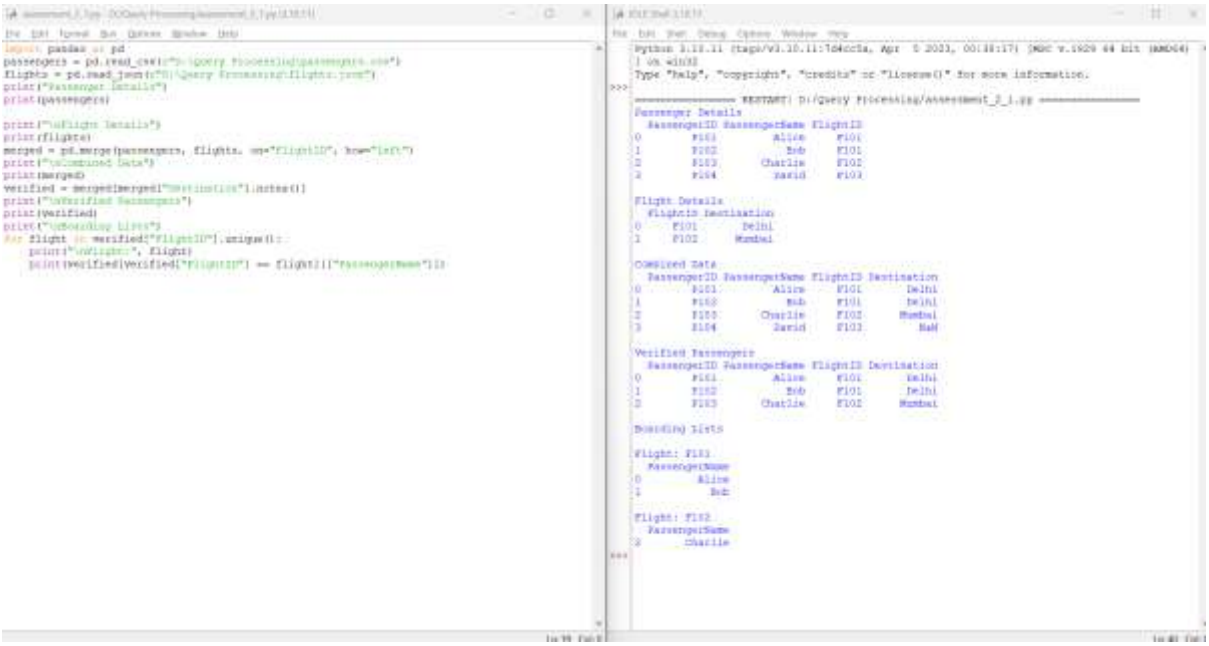
passengers.csv

P101,Alice,F101
P102,Bob,F101
P103,Charlie,F102
P104,David,F103

flights.json

```
[
  {
    "FlightID": "F101",
    "Destination": "Delhi"
  },
  {
    "FlightID": "F102",
    "Destination": "Mumbai"
  }
]
```

Output:



Result

The program successfully combines passenger and flight data, verifies bookings, and generates boarding lists for each flight.